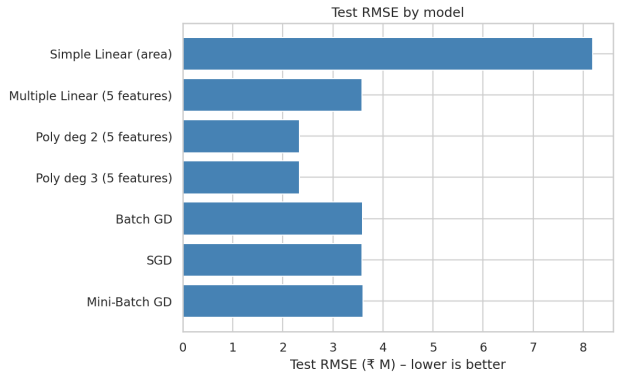
Predictive Insight Engine - Final Report

Problem: predict house prices (INR) from area, bedrooms, bathrooms, location score and age of property using supervised regression models. Dataset: 4,200 houses, no missing values or duplicates; 80/20 train-test split (3,360 / 840), random_state = 42.

1. Model comparison (test set)

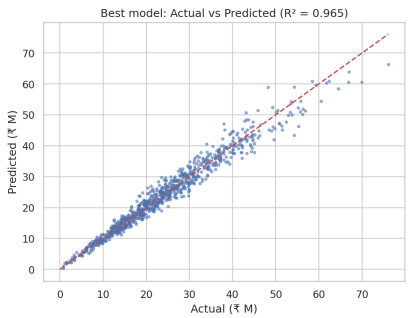
Model	Test R2	Adj. R2	MAE (INR)	RMSE (INR)
Simple Linear (area)	0.5625	0.5620	6,294,594	8,184,697
Multiple Linear (5 features)	0.9161	0.9156	2,641,320	3,584,509
Poly deg 2 (5 features)	0.9645	0.9637	1,715,219	2,330,106
Poly deg 3 (5 features)	0.9645	0.9620	1,717,780	2,331,838
Batch GD	0.9158	0.9153	2,643,293	3,589,914
SGD	0.9164	0.9159	2,697,499	3,578,545
Mini-Batch GD	0.9151	0.9145	2,645,065	3,606,576

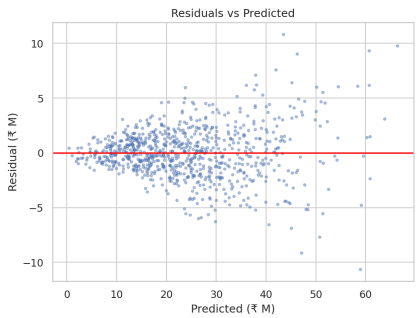


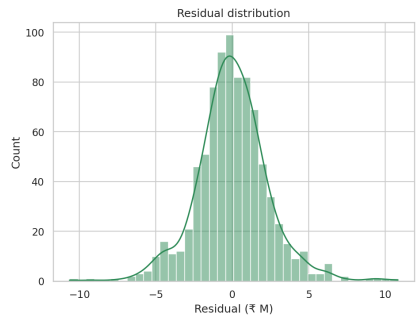


2. Best-performing model and why

Polynomial Regression (degree 2, 5 features) is best: test R2 = 0.965, RMSE = INR 2.33 M, MAE = INR 1.72 M (vs. R2 = 0.563 for Simple and 0.916 for Multiple Linear Regression). Reasons: (i) it uses all informative features; (ii) squared and interaction terms capture non-linear effects (for example size x location); (iii) the train-test gap is tiny, so variance is controlled; (iv) degree 3 gives no further test gain, so degree 2 is the simplest adequate model.





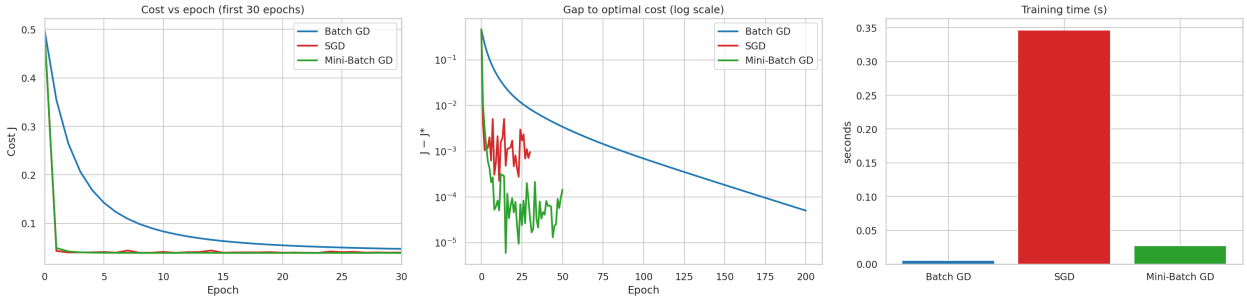


3. Impact of gradient descent optimisation

Batch, Stochastic and Mini-Batch Gradient Descent were implemented from scratch (NumPy, standardised data). All reach the same cost and test performance as the closed-form solution; GD changes how the optimum is found, not which model is best.

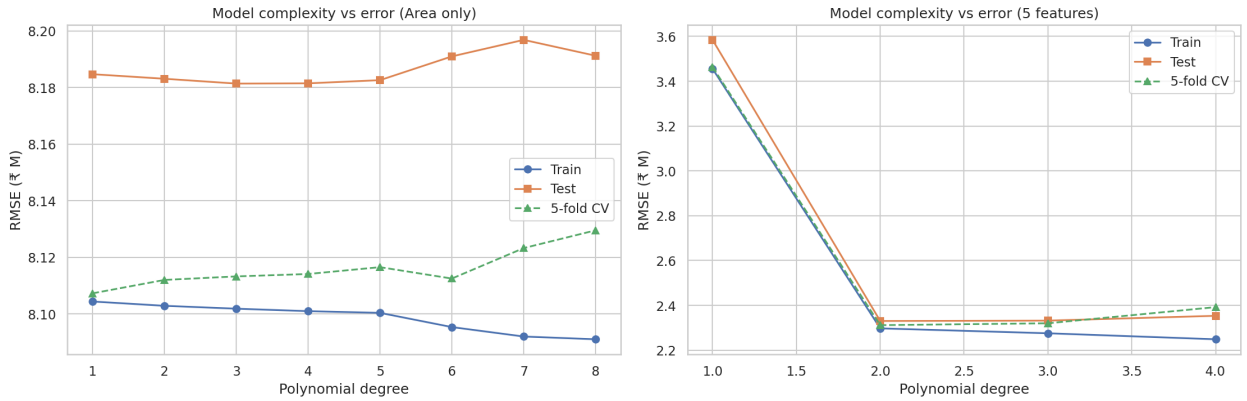
Method	Epochs to J*(1+1%)	Updates / epoch	Time (s)	Test R2
Batch GD	122	1	0.006	0.9158
SGD	8	3360	0.347	0.9164
Mini-Batch GD	6	53	0.027	0.9151

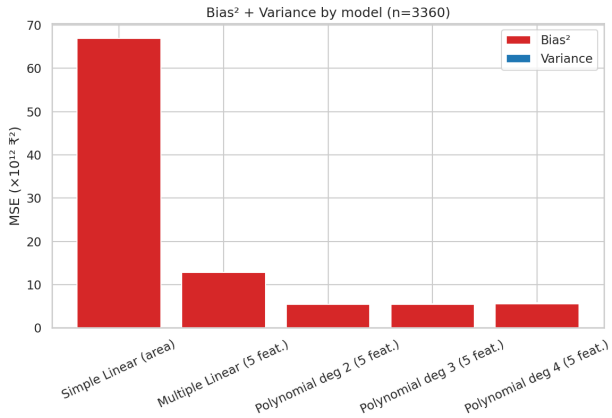
- Batch GD: smooth and stable, but one update per epoch, so it needs more epochs.
- SGD: fastest progress per epoch but noisy (does not settle exactly); slowest in wall-clock time in pure Python.
- Mini-Batch GD: best trade-off between stability and speed.
- Learning rate matters: too small is slow, too large (about 1 or more here) diverges.

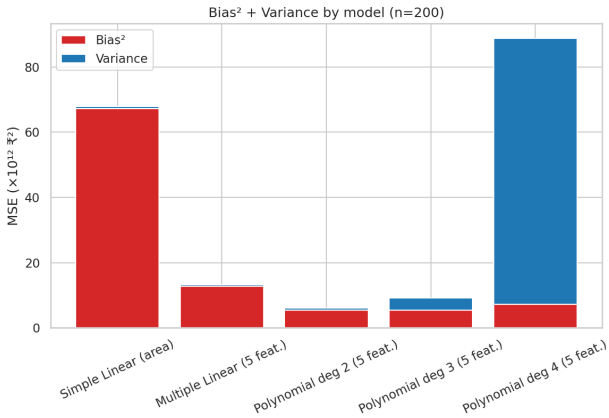


4. Evidence of overfitting / underfitting

- **Underfitting:** Simple Linear (area only): train R2 = 0.572, test R2 = 0.563; polynomial terms of area alone do not help (high bias).
- **Overfitting:** with only 150 training rows, degree 4 gets train R2 = 0.996 but test R2 = -1.78. On the full data, degrees above 2 raise the training score but not the test / CV score.
- **Bias-variance (n = 200 samples):** variance rises from 6.69e+11 (deg 2) to 8.16e+13 (deg 4) while bias does not improve.
- **Good fit:** Polynomial degree 2: train R2 = 0.966, test R2 = 0.965.







5. Practical business interpretation

- Average error of the best model is about INR 1.72 M on a mean house price of INR 23.64 M (about 7%): suitable for quick valuation, benchmarking and screening, not a replacement for a professional appraisal.
- Most valuable levers (Multiple Linear Regression, other features fixed): location score (about +INR 3.4 M per point), area (about +INR 1.4 M per +100 sq.ft), then bedrooms and bathrooms (about +INR 0.2 M each); each extra year of age lowers price by about INR 68 K.
- Limitations: errors are larger for expensive houses (heteroscedasticity); no market-timing or condition data; do not extrapolate outside 450-4,200 sq.ft.
- Next steps: log-transform the target, add Ridge/Lasso regularisation, try tree-based models and the 5 extra dataset columns.

6. Final conclusion

A single feature underfits; adding the other features gives a large jump in accuracy; degree-2 polynomial terms give the best generalisation; higher degrees only add variance. Gradient descent reaches the same optimum as the analytical solution, with Mini-Batch being the most practical. **Recommended model: Polynomial Regression (degree 2) on area, bedrooms, bathrooms, location score and age.**